

MITIT 2025 Winter Contest Editorials: Advanced Round 2

The MITIT Organizers

January 19, 2025

1 Toy Marbles

Problem Idea: Ditbul Ban

Problem Preparation: Ditbul Ban

Analysis By: Ditbul Ban

This problem is shared with Beginner Round Problem 8.

1.1 Subtask 1

Let's call the first type of action as **swap** and second type of action as **merge**.

Notice that if all c_i are distinct, then c becomes a permutation of length N . Here, we should not use the merge operation, since each container should have exactly one marble in the end.

Therefore, the problem is equivalent to sorting the permutation c using only swap operations. This is a well-known problem which can be solved using *cycle decomposition*.

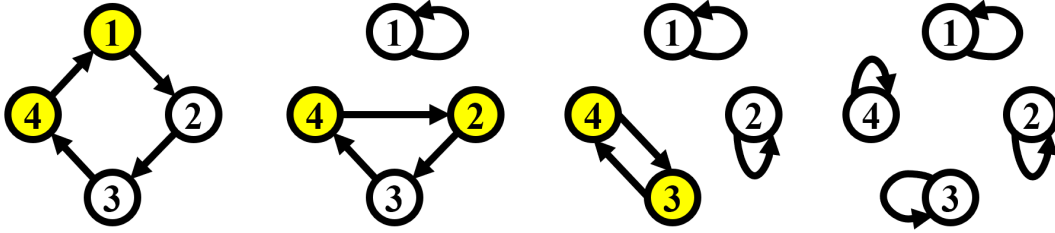
We consider the graph G , which consists N vertices and N edges. For every i s.t. $1 \leq i \leq N$, there is an edge $i \rightarrow c_i$ in the graph. (Notice that each vertex has exactly one outgoing edge; we call these graphs *functional graphs*.) Here, we can think each vertex v has an outgoing edge *pointing* where the marble in container v should actually be included. Notice that each swap operation will swap the destination of two outgoing edges in graph G ; in other words, if there were two edges $x \rightarrow e_x$ and $y \rightarrow e_y$ and we swap two containers x and y , then the two edges are removed and two new edges $x \rightarrow e_y$ and $y \rightarrow e_x$ are newly formed. Our goal is to make every edge into self loops (an edge which the starting and ending vertex is the same), since it will mean that all marbles are in their right place.

Due to the constraints of the subtask, the functional graph should be comprised only by cycles. Let's denote the initial number of cycles as s . In the final state, we should have N cycles. We could easily observe the following fact:

- Swapping two vertices in the same cycle increases the number of cycles by 1.
- Swapping two vertices *not* in the same cycle decreases the number of cycles by 1.

Obviously, we only want the number of cycles to increase, since we should make it to the maximum. Therefore, each time we should swap two vertices in the same cycle, splitting one cycle into two. Therefore we could conclude that the minimum number of operations we need is $N - s$.

After all of these observations, constructing a solution is not difficult. One simple way is iterating for every cycles, and separate all vertices in each cycle to self loops. Below is an example image of how to separate all vertices in a cycle to self loops. This can be done in $O(N)$.



1.2 Subtask 2

In this subtask, c_i are not necessarily distinct. The solution is quite intuitive, but proving the correctness is very complicated. First let's start with the following observation.

Observation 1.1. *In the optimal solution, the number of merge operations is fixed.*

Proof. Observe that each merge operation **usually** decreases the number of nonempty containers by 1. Therefore, if all merge operations are usual in this sense, then the optimal solution will contain $n - s$ merge operations, where s is the number of nonempty containers in the final state.

The unusual cases is when either x or y is empty in the merge operation. If x is empty, then this is equivalent to a swap operation, so we will consider this as a swap operation. If y is empty, then nothing happens, and we don't need this operation. Therefore, we could conclude that at least one optimal solution would not contain any unusual merge operations, and thus have exactly $n - s$ merge operations. $\square$

Therefore, the crucial part of solving this problem is to minimize the number of swap operations. We can observe one more property, which will simplify our analysis.

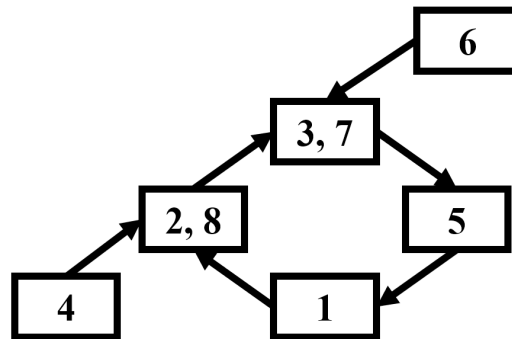
Observation 1.2. *There is an optimal solution where all the merge operations occur before all of the swap operations.*

Proof. We omit the rigorous proof here, but we will rather provide an intuition for this fact. Imagine that a merge operation happened after a swap operation. If the swap operation is independent with the merge operation (that is, they do not affect the same containers), then we can just move the merge operation to the front. If the swap operation affects a container which is also affected by the merge operation, we could accordingly change the index of the merge operation, and move it to the front. $\square$

If we consider that the merge operation happens before the swap operations, then this problem becomes very similar to Subtask 1. However, we have to decide in which container to merge the marbles of each color. To solve this problem, let's think about the following situation.

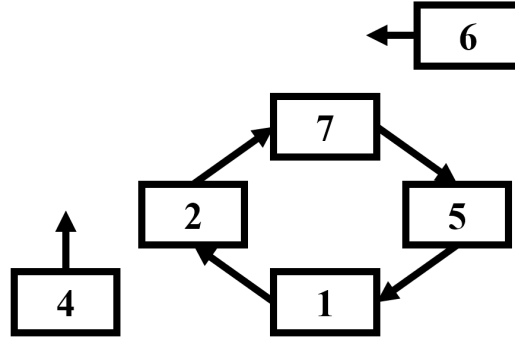
- First, partition $\{1, 2, \dots, N\}$ to one or more sets in a way such that u and v is in the same set if and only if $c_u = c_v$. We will call this partition P .
- Second, we will create a directed graph G' such that each vertex corresponds to a set in P . Each vertex will have exactly one outgoing edge, which points towards the set in P which contains the c_i value.

For example, if $N = 8$ and $c = [2, 7, 5, 8, 1, 3, 5, 7]$, the graph G will be constructed as the image below.



Notice that this graph is *similar* to the graph G in Subtask 1. The difference is that in Subtask 1, each vertex represented only one marble, but here, each vertex could represent multiple marbles. Also, recall the fact that if multiple marbles have same color, then they could be merged in any of the containers. In the above example, $c_2 = c_8 = 7$, so the two marbles can be merged in either container 2 or 8. If we decide where to merge all the marbles, then we can directly construct graph G in Subtask 1. For example, if we move

marble 8 to container 2, and move marble 3 to marble 7, we can get graph G as follows (here, marble x means the marble initially in container x):



Here, notice that vertex 4 or 6 has outgoing edges which are not pointing any vertex. This is because there are no longer any marbles in the containers they have to reach. For example, marble 4 has to move to container 8, but there are no marbles in container 8.

If we finish our decision in the merging process and get this graph G , let's figure out how many swaps we need to accomplish our goal. The graph G could be decomposed into cycles and chains which ends at a node pointing nowhere. We already analyzed the cycles in Subtask 1; we need $k - 1$ swap operations, if the cycle contains k vertices. For the chains, there are no cycles at first, but each swap operation inside the chain will increase the number of cycles by 1. This will yield a method of changing a chain of length l to l cycles in l swap operations. In conclusion, if there are $|V|$ vertices on graph G' (which should be same as the number of different colors in c), and there are k cycles in the graph, we will need $|V| - k$ swap operations to finish the process.

(For a rigorous proof, we should also analyze the cases where we perform a swap operation between cycles and chains, two different chains, or two different cycles. Analyzing each case will show that they do not do better than the methods we showed above.)

To minimize $|V| - k$, we should know about the maximum possible value of k ; in other words, how many cycles could G contain?

Claim 1.3. *The maximum number of cycles of G is equal to the number of cycles of G' .*

Proof. If we imagine the process of converting graph G' into G , it should be obvious that we cannot have more cycles in G , so this fact gives an upper bound for k .

We can also show that if we decide well on the merging process, we can construct graph G which has the same number of cycles with G' . The intuition is that we should **preserve** the cycles in G' . If a cycle in G' contains a node representing A (a set of containers),

and the node has an outgoing edge which points the node representing B (another set of containers), B will contain the destination x of all marbles in A (which is equal to the colors of all marbles in A). If we merge all marbles in B to container x , the cycle will be preserved. $\square$

Combining all these facts would give a method to use the minimum number of merge and swap operations, thus giving an optimal solution. This can be done in $O(N)$.

2 Snakes on a Grid

Problem Idea: Sam Zhang

Problem Preparation: Duru Ozer

Analysis By: Duru Ozer

This problem is shared with Beginner Round Problem 9.

2.1 Subtask 1

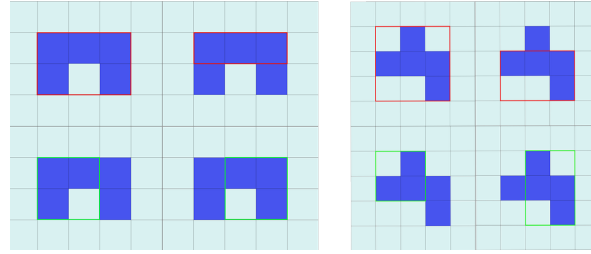
For this subtask it is enough to write a brute force solution checking whether each connected component in the queried rectangle is a snake. There are various ways to do this. One such way is to find connected components by using a DSU (Disjoint Set Union) structure. Then to check for being snakes, we could find a way to sort the cells in each component so they are in the order they would appear in a snake shape. This can be achieved by sorting the cells in the component by their coordinate pairs. Note that for a valid snake, there are two possible sortings: either both coordinates increasing / decreasing together or one coordinate increasing and the other one decreasing. We can try both of them and see if one of them works. To check if a 0-indexed sorted array of size n is a snake, it is sufficient to check for 3 things:

1. $[(r_i - r_{i-1}), (c_i - c_{i-1})] = [(r_{i-2} - r_{i-3}), (c_{i-2} - c_{i-3})]$ for $3 \leq i \leq n - 1$ i.e. The difference between adjacent cells repeats every 2 steps
2. $|r_i - r_{i-1}| + |c_i - c_{i-1}| = 1$, i.e, the distance between each adjacent cell pair is 1
3. $[(r_1 - r_0), (c_1 - c_0)] = [(r_2 - r_1), (c_2 - c_1)]$, i.e. The component is not a line (only if $n > 2$)

So we can try sorting the cells in both ways and checking for these 3 conditions. If all 3 of them hold for at least one sorting, the component is a snake. Total time complexity : $O(A \log A)$, where A is the sum of areas over all queries.

2.2 Subtask 2 & 3

The key observation behind subtasks 2 and 3 is that for every non-snake component, there are $O(1)$ rectangles partially or fully contain that component and include a non-snake piece of it. In the following figure, such rectangles are marked by red.



We can find all such rectangles for each component in the grid in $O(1)$, thus total complexity over the grid being $O(NM)$. Once we find them, a queried rectangle is good if and only if it doesn't contain any of the non-snake rectangles. We can check this by using 2D prefix sums. Specifically, we can use a different prefix sum for different rectangle sizes. For example, for Subtask 2, these sizes would be $[(1, 4), (2, 2), (2, 3), (3, 2), (2, 2), (4, 1)]$. Note that the number possible sizes for the defined rectangles for both subtasks is constant. Thus the total complexity is $O(NM + Q)$ with the constant differing between subtasks.

2.3 Full Solution

The key observation to the full solution is that, if a component is not a snake, it either contains 3 cells in a row or 4 cells in a 2×2 square. This can easily be checked in $O(1)$ by using 3 2D prefix sums:

1. One for checking 3 horizontally adjacent cells with the same value
2. One for checking 3 vertically adjacent cells with the same value
3. One for checking 2×2 squares with the same value

Namely, for each of the 3 unwanted shapes, we can choose a designated cell to mark, for instance the middle cell for the 3-in-a-row cases and the top left corner for the 2×2 square case. Then we can mark those cells wherever such a shape exists, and adjust the boundaries for the prefix sum so that if a cell is marked and is inside the boundaries, the unwanted shape is present in the queried rectangle.

3 MIT Tour

Problem Idea: Andrii Smutchak

Problem Preparation: Andrii Smutchak

Analysis By: Andrii Smutchak

The problem can be represented as weighted rooted tree, where the rooms are vertexes and corridors are edges.

Define $depth_v$ as the number of edges between root and vertex v in original graph.

3.1 Subtask 1

The first critical observation is that the answer can be calculated using dynamic programming. For each vertex v , we define dp_v as minimal cost of sequence that ends with v . We can calculate dp_v level by level by the following formula:

$$dp_v = \begin{cases} \min_u dp_u + dist(u, v) & \text{if } depth_v > 1 \\ dist(1, v) & \text{otherwise.} \end{cases}$$

where the minimum in the first case is taken over all nodes u on the level above v , excluding v 's parent. The final answer will be $\min dp_v$, for vertexes on the maximum depth.

We can naively implement this to work in $O(N^2)$.

3.2 Subtask 2

The solution from subtask 1 can be optimized. The key observation is that path between each two vertexes can be splitted into upward and downward parts. Instead of iterating through each pair of vertices u, v on adjacent levels, we will save a value tmp_l to be defined later for each parent l of u that will aid us in computing all dp_v . So, suppose we already calculated dp_u for every vertex on level $k - 1$.

- Define $tmp_l = \min_u dp_u + dist(u, l)$, where u is in subtree of l and $depth_u = k - 1$. We may calculate tmp_l for every vertex above level $k - 1$ starting from vertices on level $k - 1$ working towards the root.
- We can now go from root to leaves and calculate dp_v by pushing values downward. From each vertex l , let's say the value w was received from its parent. Then, we push the value $\min(w, tmp_l) + dist(u, l)$ to each child u of l . We can see that the correct value of dp_v gets pushed to each node v on level k the value of dp_v .
- Notice that we need to enforce that we do not include parent-child transition in the dp, and that we only ever use simple paths. To do this, we can instead augment tmp_l to store two minimum values and the children from which they are calculated.

Then we can push down the value only from others children, to exclude loops and parent-child transition.

Using this approach we run a DFS twice for each level, so the total runtime is $O(N \cdot \text{depth})$.

3.3 Subtask 3

The last critical observation is that the tree can be compressed. Suppose we trying to recalculate the dp from level k to level $k + 1$. If a vertex on level k has only one child, it can be deleted and the child can be directly connected to the vertex's parent with an edge that has the sum of the distance of the two.

Claim 3.1. *If we do the optimization described above, the total runtime will be $O(N)$.*

Proof. The complexity for computing the dp_v values for level k is proportional to the number of vertices that are left at or above level k . By the optimization, every vertex above level k has at least two children, so the total number of vertices above level k is at most $2 \cdot (\# \text{ vertices on level } k) = O(\# \text{ vertices on level } k)$.

The total time will be $O(\sum \# \text{ vertices on each level}) = O(N)$. □

4 Path Partition

Problem Idea: Zixiang Zhou

Problem Preparation: Zixiang Zhou

Analysis By: Zixiang Zhou

Test cases 1 and 2 have $M = \frac{N(N-1)}{2}$, a complete graph. For test case 1, the degree of every node is even, and one solution is to take an Eulerian tour and cut it into pieces of length 3. For test case 2, the degree of every node is odd. One solution is to first pair up the nodes with disjoint paths of length 3, and then use up the remaining edges by cutting up an Eulerian tour as before. Many more explicit constructions that do not require algorithmically finding an Eulerian tour are possible.

The remaining test cases have the same M and increasing values of N . Following the ideas from test cases 1 and 2, our approach will be to pair up the odd-degree nodes by disjoint paths of length 3, and afterwards, take an Eulerian tour on the remaining graph and cut it into pieces of length 3. This might cause problems if the remaining graph becomes disconnected with components whose number of edges is not a multiple of 3, but we can hope this won't happen in practice since M is much larger than N .

One heuristic is to greedily pair up odd-degree nodes as follows: For each edge connecting two even-degree nodes u and v (in an arbitrary order), loop through their neighbors. If we find an odd-degree node u' adjacent to u and an odd-degree node v' adjacent to v , then use path $u' - u - v - v'$. By only considering even-degree u and v , it's impossible to generate overlapping paths this way. This approach can solve test cases 3 and 4.

We can improve this approach by also allowing u and/or v to be odd-degree, taking some care to delete edges once they are assigned. This approach can solve up to around test case 5.

Some further improvements to this heuristic include prioritizing smaller degree odd nodes to be used as u' and/or v' . It may be possible to pass more than test case 5, though we anticipate it will be difficult since test case 6 has three times larger N than test case 5.

For test cases 7-9, greedily forming a matching on odd-degree nodes is very unlikely to succeed because the graph is too sparse; it's likely we create "isolated" nodes in the process. Instead, we can try to find a *spanning tree* on the odd-degree nodes. Afterwards, we DFS this spanning tree to find a subset of edges such that every node in the the subgraph formed is connected to an odd number of other nodes.

To pass the final test case, the heuristic needs to be very well optimized, incorporating ideas such as prioritizing low-degree nodes u' and v' , prioritizing edges u and v where 2/1/0 of the endpoints have even degree, and especially prioritizing nodes of degree 1.

Remarks: It can be proven that determining whether a graph can be partitioned into paths of length 3 is NP-complete. For an Erdos-Renyi graph $G(n, p)$, the threshold for having no isolated edges is sharp at $p = \frac{\ln(n)}{2n}$ (see <https://math.stackexchange.com/questions/4583114/limit-probability-for-a-random-graph-to-have-an-isolated-edge>). The final test case has $M = 300000$ quite close to $\frac{N(N-1)}{2} \cdot \frac{\ln(N)}{2N} = \frac{(N-1)\ln(N)}{4} \approx 256667$, and it contains several isolated nodes but no isolated edges. We conjecture that for large n , with high probability, $G(n, p)$ will be partitionable into paths of length 3 as long as the number of edges it contains is a multiple of 3 and there are no isolated edges.

5 Colored Blocks

Problem Idea: Anton Trygub

Problem Preparation: Benjamin Chen

Analysis By: Anton Trygub

For an array arr and number x , define $bal(arr, x)$ as $2 \times$ number of occurrences of x in arr minus the length of arr . The intuition for this notion is: if we replace all elements equal to x with 1, and all elements different from x with -1 , $bal(arr, x)$ is the sum of elements of the resulting array.

Claim 1. The number of needed lines has to be at least $bal(c[l : r], x)$ for every subarray $c[l : r]$ and every element x .

Proof. Consider any split into t subsequences that satisfies the conditions. Then $c[l : r]$ was split into some subsequences, balance of each being ≤ 1 . Therefore, $bal(c[l : r], x) \leq t$.

We claim that it's actually sufficient: if T is the maximum value of $bal(c[l : r], x)$ over all $c[l : r]$ and x , then we can split everything into T subsequences. In fact, we will show even a bit more general statement:

Claim 2. Assume we already split $c[1 : k]$ into T subsequences satisfying the conditions (some of them might be empty), and let $cnt[x]$ denote the number of these subsequences ending with x . For each value x denote $pref_bal[x]$ as the largest value of $bal(c[k+1 : r], x)$ over all $k+1 \leq r \leq N$. If $cnt[x] + pref_bal[x] \leq T$ for every x , then it's possible to split the remaining elements appropriately.

Proof. Clearly, this condition is true at the beginning for every x . We will show that if condition holds now, we can assign c_{k+1} to one of T subsequences so that the conditions hold on the next step as well.

First, we can definitely append c_{k+1} at least to some subsequences: if all of them end with c_{k+1} currently, we have $cnt[c_{k+1}] + pref_bal[c_{k+1}] \geq T + 1$, contradiction. Now, let's see how these values will change on the next step, after we append c_{k+1} somewhere:

- $pref_bal[c_{k+1}]$ will decrease by 1.
- For every other x with $pref_bal[x] \geq 0$, $pref_bal[x]$ will increase by 1.
- $cnt[c_{k+1}]$ decreases by 1
- If we append c_{k+1} to subsequence ending with y , $cnt[y]$ decreases by 1.

So, the condition will clearly hold for c_{k+1} on the next step, and if for some $x \neq c_{k+1}$ we have $cnt[x] + pref_bal[x] = T$, we **have to** append c_{k+1} to a subsequence ending with x , to decrease $cnt[x]$. This is sufficient to maintain the invariants, the only concerning

point is: if there are at least two such elements x , we are doomed: we cannot fix two such elements at a time. Fortunately, this can never happen. To show this, we will show another claim.

Claim 3. For any $x, y \neq c_{k+1}$ we have $\text{pref_bal}[x] + \text{pref_bal}[y] < T$.

Proof. Assume $\text{pref_bal}[x] = \text{bal}(c[k+1 : r_x], x)$, $\text{pref_bal}[y] = \text{bal}(c[k+1 : r_y], y)$ with $x < y$. Clearly $\text{pref_bal}[x] \leq T-1$, since otherwise $\text{bal}(c[k+2 : r_x], x) = \text{bal}(c[k+1 : r_x], x) + 1 > T$, same for $\text{pref_bal}[y]$. Also, denote $\text{len}_x = r_x - k$, $\text{len}_y = r_y - k$.

Then there are at least $\frac{\text{len}_x + \text{pref_bal}[x]}{2}$ elements equal to x on $c[k+1 : r_x]$, and one element on this subarray is equal to c_{k+1} . Hence, balance of y on $c[k+1 : r_x]$ is at most $-\text{pref_bal}[x] - 2$. But then balance of y on $c[r_x+1 : r_y]$ has to be at least $\text{pref_bal}[x] + 2 + \text{pref_bal}[y]$, so this quantity cannot exceed $T-2$. $\square$

From this claim it already follows that $(\text{cnt}[x] + \text{pref_bal}[x]) + (\text{cnt}[y] + \text{pref_bal}[y]) = (\text{cnt}[x] + \text{cnt}[y]) + (\text{pref_bal}[x] + \text{pref_bal}[y]) \leq T + (T-2)$, so this cannot equal $2T$.

This finishes the prove of **Claim 3**. Now we just need a way to implement this. Fortunately, it's pretty easy to maintain all $\text{pref_bal}[x]$ s, because there are only a few of them at each point! Let's prove this

Claim 4. Assume there are t elements $x_1, x_2, \dots, x_t$, such that $\text{bal}(c[1 : \text{len}_i], x_i) \geq 0$ for all i , with $\text{len}_1 \leq \text{len}_2 \leq \dots \leq \text{len}_t$. Then $t = O(\log N)$.

Proof. Note that there are at least $\frac{\text{len}_i}{2}$ elements equal to x_i on $c[1 : \text{len}_i]$. So, we need at least $\sum_{j=1}^{i-1} \frac{\text{len}_j}{2}$ elements equal to x_i on $c[1 : \text{len}_i]$ to make its balance nonnegative, which implies that $\text{len}_i \geq \text{len}_1 + \text{len}_2 + \dots + \text{len}_{i-1}$. This implies that $\text{len}_i \geq 2^{i-1}$ by induction.

So, here is how we are going to implement it. First, we traverse sequence from end to beginning, calculating all elements with nonnegative $\text{pref_bal}[x]$ for each suffix and corresponding pref_bals . We can do this as follows: once we process next element, we just find its own pref_bal , and update all current pref_bals (just decrease each of them by 1).

With this information, we can do our algorithm from **Claim 2**. At each step, we just need to find whether there is x such that we need to decrease $\text{cnt}[x]$, to append to a subsequence ending with x . Otherwise, we can append to any subsequence. This gives a final runtime of $O(N \log N)$.